College Club Event Management System

DBMS Individual Evaluation & Viva Survival Guide | Academic Year 2026

Subject: Database Management Systems (DBMS)	**Target Stack:** Oracle XE 21c / Python Flask
Student Role: Second Year Engineering Student	**Prepared For:** One-on-One Faculty Review & Viva

Part 1: The 1-Minute Pitch (What to Say When You Sit Down)

When your faculty asks: *'Tell me about your project'*, do not panic or ramble. Speak this exact, confident answer:

> *"Good morning / afternoon Ma'am. My project is the **College Club Event Management System**.*
>
> *In our college, various technical, cultural, and sports clubs organize events throughout the semester. Usually, registrations and member lists are handled using scattered Google Forms or paper sheets. This causes major issues like duplicate signups, no membership verification, and students accidentally registering for past events.*
>
> *To solve this, I designed a **normalized relational database in 3NF** with 5 core tables. I implemented strict database integrity constraints, **two database views** for reporting, a **PL/SQL stored procedure** that blocks duplicate or expired registrations, and a **database trigger**.*
>
> *On top of this, I built a clean web interface using **Python Flask** and parameterized SQL queries to prevent SQL injection. May I demonstrate how the system works?"*

Part 2: How the System Works (Architecture in Plain English)

Your system follows a clean 3-Tier Architecture that you can explain on the whiteboard:

Tier	Technology	What It Does in Simple Words
Frontend (Presentation)	HTML5, Vanilla CSS3, Jinja2	Displays web pages (Home, Students, Clubs, Events, Reports). Takes user inputs via forms. No heavy JS frameworks.
Backend (Application)	Python 3 + Flask (app.py)	Receives HTTP requests, runs parameterized SQL queries using bind variables (:param), commits transactions, and renders templates.
Database (Storage & Logic)	Oracle Database 21c XE (with SQLite local fallback)	Stores tables in 3NF, executes views for fast analytical lookups, runs stored procedures for registration logic, and fires triggers.

Part 3: Step-by-Step Live Demo Guide (What to Click & What to Say)

Follow this exact 5-step sequence when demonstrating on your laptop:

Step 1: The Home Page (http://127.0.0.1:5000)

- **Action:** Show the 3 top metric cards (Students, Clubs, Upcoming Events) and the upcoming events table.
- **What to Say:** *'Ma'am, this dashboard automatically fetches live metrics. The Upcoming Events table filters events dynamically using WHERE event_date >= SYSDATE so expired events are never shown to students.'*

Step 2: Students Page (/students) — Adding a Student

- **Action:** Enter a name (e.g. 'Varun Kumar'), email ('varun@college.edu'), department ('Computer Science'), year ('2'). Click **Add Student**.
- **What to Say:** *'Here we have our Student table. The input year is strictly restricted between 1 and 4 using a domain CHECK constraint, and emails must be unique.'*

Step 3: Clubs Page (/clubs) — Database View & Joining

- **Action:** Point out the member count. Click **'View Members'** on Coding Club. Then enroll Varun into Robotics Society using the join form.
- **What to Say:** *'This member list is powered by our database view **club_member_list** which pre-joins CLUB, MEMBERSHIP, and STUDENT. When joining, the composite constraint UNIQUE(student_id, club_id) ensures a student cannot join the same club twice.'*

Step 4: Events Page (/events) — Stored Procedure & Duplicate Block

- **Action:** Try registering Aarav Sharma for 'CodeStorm' (already registered) -> Show red error message. Then register Varun Kumar -> Show green success message. Finally, click **Cancel** on any registration.
- **What to Say:** *'Registration is handled by our PL/SQL stored procedure **register_student_for_event**. Notice how it blocked Aarav because he was already registered, but allowed Varun. It returns a clean status message directly from the database.'*

Step 5: Reports Page (/reports) — Showcase Advanced SQL

- **Action:** Scroll through the 4 analytical report tables.
- **What to Say:** *'This page demonstrates complex SQL queries required in the DBMS curriculum: multi-table INNER and LEFT JOINs, GROUP BY, aggregate functions like COUNT(), and the **HAVING >= 2** clause to filter grouped records.'*

Step 6: Show the Command-Line Interface (view_db.exe)

- **Action:** Open CMD, run view_db.exe, press **4** to show memberships.
- **What to Say:** *'Ma'am, our database is fully persistent and synchronized. Any member I just added in the web browser is immediately visible here in the SQL console.'*

Part 4: Database Schema & Entity-Relationship (ER) Design

The database consists of **5 tables** designed strictly from your ER Diagram:

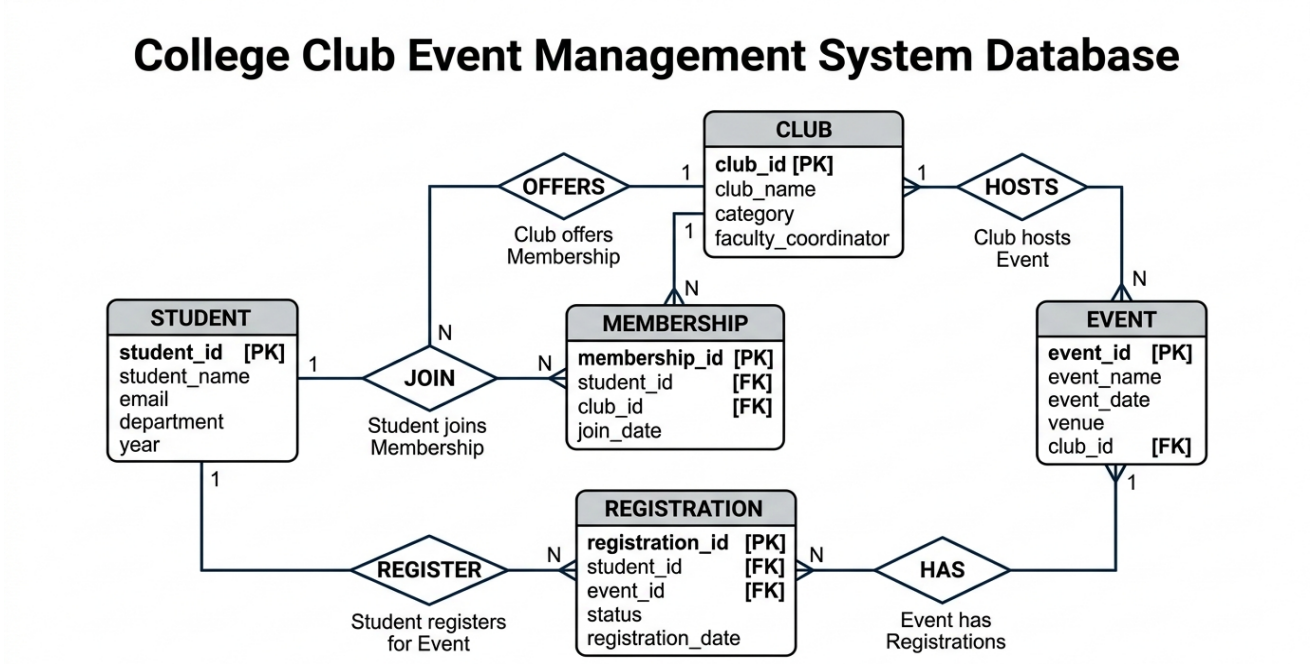


Table	Primary Key (PK)	Foreign Keys (FK)	Key Constraints
STUDENT	student_id	None	UNIQUE(email) CHECK(year BETWEEN 1 AND 4)
CLUB	club_id	None	UNIQUE(club_name)
EVENT	event_id	club_id → CLUB(club_id)	ON DELETE CASCADE
MEMBERSHIP	membership_id	student_id → STUDENT club_id → CLUB	UNIQUE(student_id, club_id) ON DELETE CASCADE
REGISTRATION	registration_id	student_id → STUDENT event_id → EVENT	UNIQUE(student_id, event_id) CHECK(status IN ('REGISTERED', 'CANCELLED', 'ATTENDED'))

Part 5: Normalization Justification (How to Prove 3NF)

If the faculty asks: *'Prove your database is normalized up to 3NF'*, explain these 3 clear points:

1. First Normal Form (1NF) — Atomic Values

- Every column in every table contains single, indivisible (atomic) values.
- There are no repeating groups (e.g. we do not store multiple events in one column as 'Event1, Event2').
- Every table has a designated Primary Key.

2. Second Normal Form (2NF) — No Partial Dependency

- The database is already in 1NF.
- No non-key attribute depends on a part of a composite primary key. Every non-key column depends on the full primary key.

- In associative tables (MEMBERSHIP and REGISTRATION), surrogate primary keys (membership_id, registration_id) are used, so all attributes depend on the full primary key.

3. Third Normal Form (3NF) — No Transitive Dependency

- The database is already in 2NF.
- No non-key column determines another non-key column (no X → Y where X is non-prime).
- **Concrete Example:** In the EVENT table, we only store club_id. We do NOT duplicate club_name or faculty_coordinator in EVENT. Those details belong exclusively in CLUB.
- **Concrete Example 2:** In REGISTRATION, we only store student_id and event_id. We do not store student_name or email in REGISTRATION.

Part 6: Advanced DBMS Features Implemented

1. View 1: event_registration_count

Combines EVENT, CLUB, and REGISTRATION using a LEFT JOIN and GROUP BY. Counts how many students are currently registered for each event. Simplifies coordinator queries without calculating aggregates repeatedly.

2. View 2: club_member_list

Combines CLUB, MEMBERSHIP, and STUDENT using INNER JOIN. Provides a complete student roster for each club, abstracting away the complex join logic from web pages.

3. Stored Procedure: register_student_for_event

Accepts p_student_id, p_event_id, and returns p_message (OUT).
- Checks if student and event exist.
- Validates event date: If event_date < SYSDATE, rejects registration.
- Checks duplicate registration: If already registered, rejects signup.
- If previously cancelled, re-activates status to 'REGISTERED'. Otherwise inserts fresh row.

4. Trigger: prevent_past_event_registration

A BEFORE INSERT ON REGISTRATION FOR EACH ROW trigger. If any application or user tries to insert a registration for an event whose date has already passed, the trigger executes RAISE_APPLICATION_ERROR(-20001, 'Registration Closed'). This guarantees database-level integrity even outside the web UI.

Part 7: 15 Must-Know Viva Questions & Instant Answers

Read these questions and short answers before walking into the room:

Q1: What are the main entities in your ER diagram and what do they represent?

We have 5 entities: STUDENT (student profile), CLUB (student organizations), EVENT (activities hosted by clubs), MEMBERSHIP (students joining clubs), and REGISTRATION (students signing up for events).

Q2: What is an associative entity and why did you use it?

An associative entity resolves a Many-to-Many (M:N) relationship. A Student can join many Clubs, and a Club has many Students. We cannot store this directly in one table without repeating data, so we created the associative entity MEMBERSHIP.

Q3: What is the difference between a Primary Key and a Unique Key?

A table can have only one Primary Key, but multiple Unique keys. Primary Key cannot contain NULL, while Unique keys allow NULL (unless marked NOT NULL).

Q4: What is Referential Integrity?

Referential integrity ensures relationships between tables remain consistent. Foreign keys cannot reference a non-existent parent primary key. In our schema, an event cannot reference a club_id that does not exist.

Q5: What does ON DELETE CASCADE do in your schema?

When a parent record is deleted, all matching child records are automatically removed. For example, deleting a club automatically deletes all events hosted by that club and its memberships, preventing orphan rows.

Q6: What is a Database View and what is its advantage?

A View is a virtual table based on a saved SQL query. It does not duplicate physical data. It provides query simplicity, security (restricting direct table access), and abstraction of complex JOINs.

Q7: What is the difference between a Stored Procedure and a Function?

A Stored Procedure does not have to return a value and can have IN, OUT, and IN OUT parameters. A Function must return a single value and can be called directly inside a SQL SELECT statement.

Q8: What is the difference between a Stored Procedure and a Trigger?

A Stored Procedure must be explicitly invoked by an application or user (e.g. cursor.callproc). A Trigger executes automatically (fires) when a DML event like INSERT, UPDATE, or DELETE occurs on a table.

Q9: What is the difference between WHERE and HAVING clauses?

WHERE filters individual rows BEFORE grouping. HAVING filters groups AFTER aggregate functions (like COUNT, SUM) have been calculated in a GROUP BY clause.

Q10: What is the difference between INNER JOIN and LEFT JOIN?

INNER JOIN returns only rows that have matching keys in both tables. LEFT JOIN returns all rows from the left table, and matching rows from the right table (filling NULL if there is no match).

Q11: Why did you use SQL bind variables (:param) instead of string formatting?

Two reasons: 1. Security: Completely blocks SQL Injection attacks because input is treated purely as data, not executable code. 2. Performance: Allows Oracle to reuse compiled execution plans in the shared pool.

Q12: What CHECK constraints did you implement in your project?

We implemented: 1. `CHECK (year BETWEEN 1 AND 4)` on STUDENT to prevent invalid academic years. 2. `CHECK (status IN ('REGISTERED', 'CANCELLED', 'ATTENDED'))` on REGISTRATION to enforce valid status values.

Q13: What is a composite UNIQUE constraint and where is it used?

A composite UNIQUE constraint enforces uniqueness across a combination of multiple columns. In MEMBERSHIP, `UNIQUE(student_id, club_id)` prevents a student from joining the same club twice.

Q14: What are ACID properties in database transactions?

Atomicity (all or nothing), Consistency (preserves constraints), Isolation (concurrent transactions do not interfere), and Durability (committed data survives server crashes). Every transaction in our app uses commit() or rollback().

Q15: How does your application connect to Oracle Database?

We use python-oracledb in pure Thin Mode. It connects directly over TCP/IP to localhost:1521/XEPDB1 without requiring Oracle Instant Client or external C libraries.

Part 8: Faculty 'Trap' Questions & Confident Answers

Evaluators love testing your conceptual depth with these tricky questions:

Trap 1: 'Why didn't you just add club_name into the EVENT table to make queries faster?'

Answer: "Ma'am, storing club_name in EVENT would introduce a **transitive dependency** (event_id → club_id → club_name), which violates Third Normal Form (3NF). If a club's name is ever updated, we would have to update multiple event records, creating an **update anomaly**. In 3NF, club_name stays strictly in the CLUB table."

Trap 2: 'If you already have a stored procedure checking the event date, why do you need a trigger?'

Answer: "Ma'am, the stored procedure protects registrations coming from our web app. But if a database administrator or another application directly executes an INSERT in SQL*Plus or DBeaver, the procedure is bypassed. The **trigger acts as a database-level safety net** that enforces the rule regardless of which client connects."

Trap 3: 'What happens if two students try to register at the exact same millisecond?'

Answer: "Our database uses **Isolation and unique constraints**. The constraint UNIQUE(student_id, event_id) ensures that even under concurrent requests, only one insert can succeed while the second will be rejected by the database engine without data corruption."